

Week13_예습과제_이재린

태그

BERT

BERT

: 트랜스포머 기반 양방향 인코더를 사용하는 언어 처리 모델

- 양방향 인코더 : 입력 시퀀스를 양쪽 방향에서 처리 >> 이전과 이후 단어 모두 참조하여 더 정확하게 단어의 의미와 문맥 파악 가능
- 이미 대규모 데이터를 사용해 사전 학습되어있음 & 전이 학습에 주로 활용 OR 재사용하여 적은 양의 data로도 높은 정확도 달성 가능

[트랜스포머의 인코더를 기반으로 하는 BERT 모델]

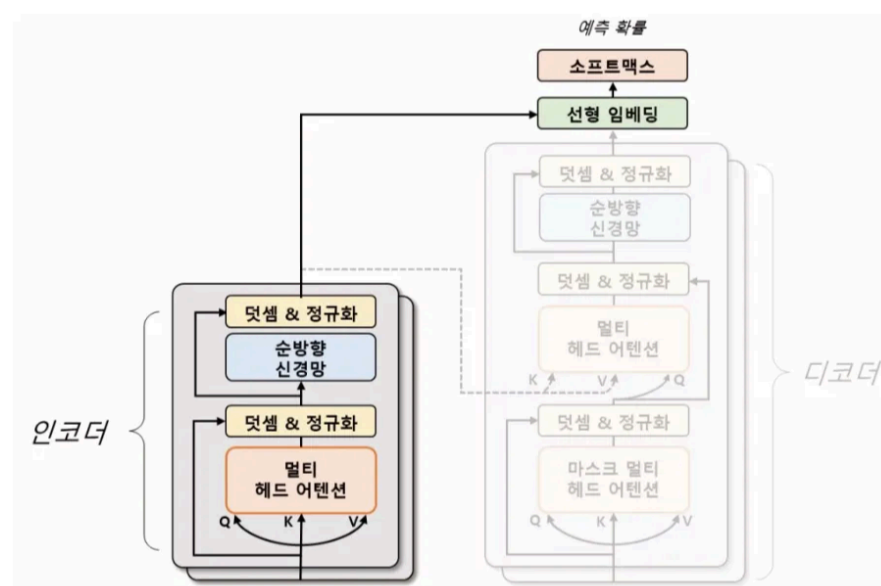


그림 7.13 트랜스포머와 BERT 모델 비교

- 인코더 모듈만 사용 >> 입력 문장의 단어들을 임베딩 & 의미를 벡터화 >> 순차적으로 처리하여 문장 전체의 의미를 추출
- 사전 학습으로 마스킹된 언어 모델링 MLM과 다음 문장 예측 방법 NSP 사용

사전 학습 방법

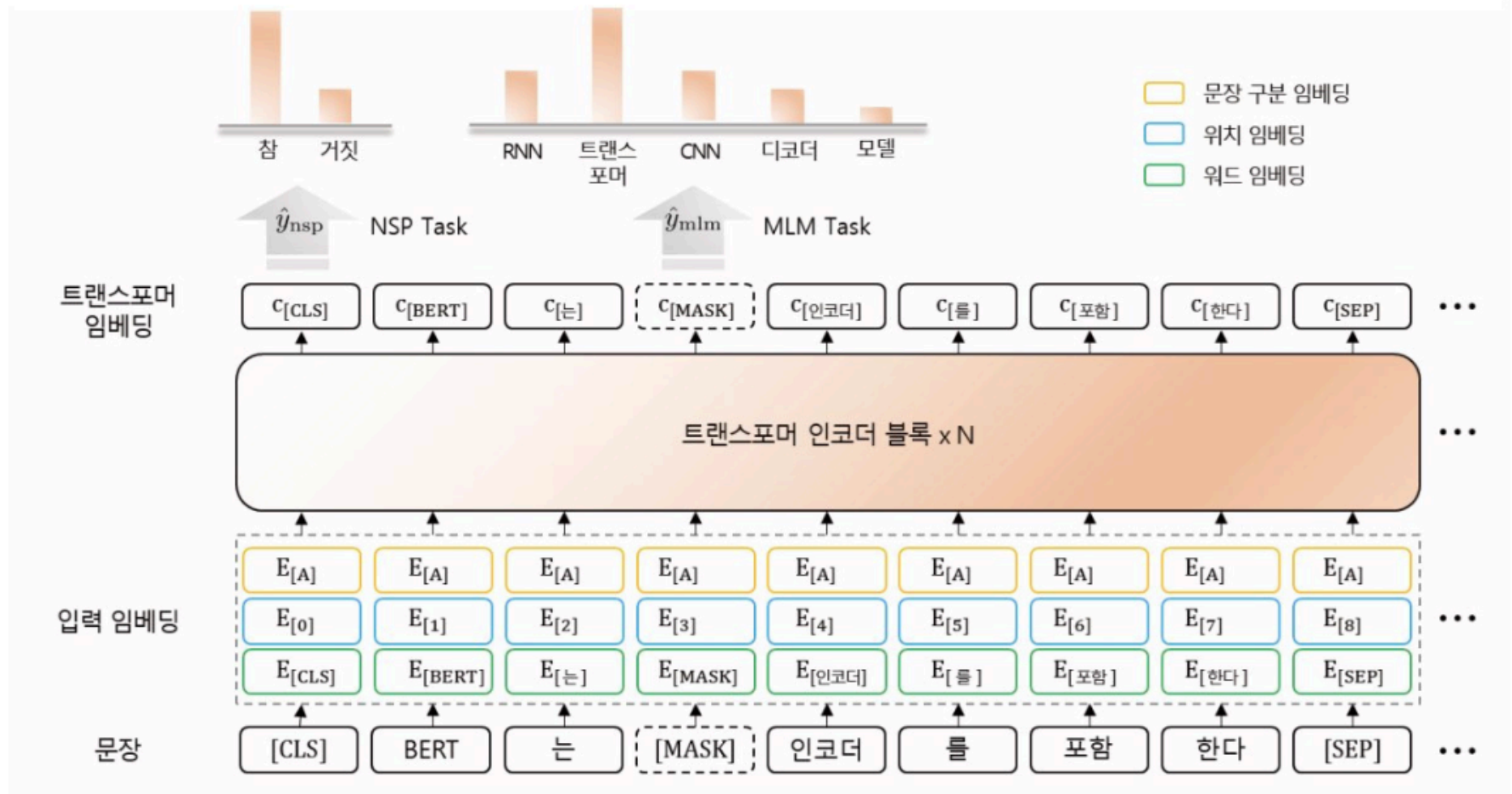
1. MLM과 NSP

- MLM : 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식, 양방향 문맥 정보 참고
 - ex) I'm learning Pytorch(learning을 마스킹) → I'm [MASK] Pytorch
- NSP : 두 개의 문장을 받았을 때 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단
 - BERT는 두 문장의 연속 관계 예측 및 관계 학습&의미적 유사성 파악

2. 토큰 : 특수 토큰을 추가해 모델의 학습과 추론에 정보를 제공

- [CLS] : 입력 문장의 시작 부분에 추가되는 토큰 → 입력 문장의 유형 미리 파악 가능
- [SEP] : 두 개의 문장을 구분할 때 사용하는 토큰
- [MASK] : 임의로 선택된 단어를 나타내는 토큰 → 단어 예측에 활용

⇒ [CLS] 문장1 [SEP] 문장2 [MASK] [SEP] 이런 형태



- BERT는 사전 학습 후 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용 가능

모델 실습

1. Korpora 라이브러리로 데이터 불러오기
2. BertTokenizer 클래스로 데이터 전처리
 - do_lower_case=False : 대소문자 구분 X
 - RandomSampler : 무작위로 샘플링 > 학습에 적용 & 샘플러 클래스로 데이터 샘플링
 - SequentialSampler 데이터를 고정된 순서대로 반환 > 검증 및 평가 배치
3. BertForSequenceClassification로 모델 선언
 - 토큰 임베딩 + 위치 임베딩 + 문장 구분 임베딩으로 구성

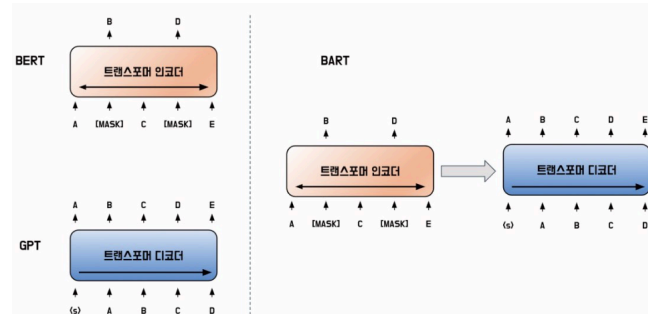
```
bert
├── embeddings
│   ├── word_embeddings
│   ├── position_embeddings
│   ├── token_type_embeddings
│   ├── LayerNorm
│   └── dropout
├── encoder
│   ├── layer
│   │   ├── 0
│   │   ├── 1
│   │   ├── 2
│   │   ├── 3
│   │   ├── 4
│   │   ├── 5
│   │   ├── 6
│   │   ├── 7
│   │   ├── 8
│   │   ├── 9
│   │   ├── 10
│   │   └── 11
│   └── pooler
│       ├── dense
│       └── activation
└── dropout
    └── classifier
```

4. 모델 학습 및 평가

BART

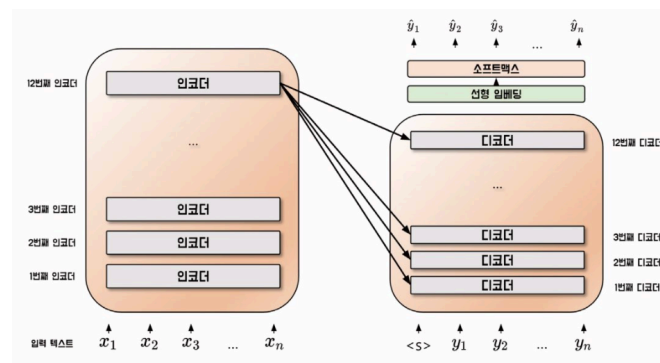
BART

- BART : BERT의 인코더 + GPT 디코더 구조인 **시퀀스-시퀀스 구조**
- 노이즈 제거 오토인코더로 사전 학습 → 입력 데이터에 잡음을 추가 & 잡음없는 원본 복원하는 학습
- BERT의 인코더
 - 일부 단어를 무작위로 마스킹 → 마스킹된 단어 맞추면서 문장 전체 맥락 이해 & 단어의 상호작용 in 문맥 파악
- GPT의 디코더
 - 이전 토큰을 입력받아 다음 토큰 예측



- 노이즈 제거 오토 인코더로 노이즈 추가 & 복원하면서 학습 = BERT 인코더+GPT 디코더
- BART는 이를 통해 문장 구조와 의미를 보존하면서 다양한 변형 학습 ㄱㄴ

[모델 구조]



- 인코더와 디코더 모두 사용 → 트랜스포머와 유사한 구조 but 인코더의 마지막 계층(입력 문장 전체의 의미를 잘 반영하는 벡터)과 디코더 (앞의 벡터를 참고)의 각 계층 사이에만 어텐션 연산 수행 → 정보 전달 & 메모리 용량 줄이기 ㄱㄴ

사전 학습 방법

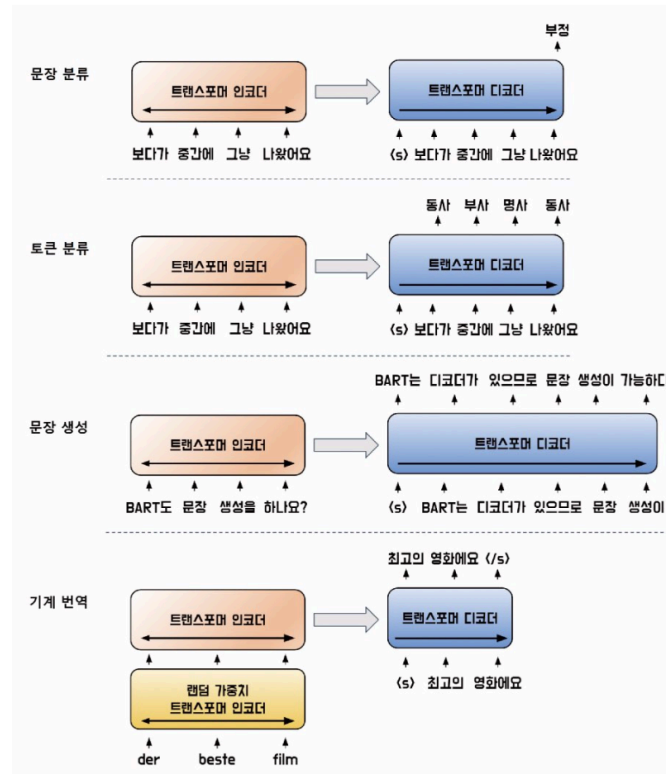
MLM이외의 다양한 노이즈 기법 사용

원본 문장	노이즈 기법
그는 정문으로 발을 옮겼다. 그의 아내는 멀어지는 그를 바라보고 있었다. 그녀는 눈물을 흘렸다. 그도 마찬가지였다.	토큰 마스킹 그는 . 발을 옮겼다. 아내 는 멀어지는 그를 . 있었다. 그녀는 눈물을 . 그도 .었다. 토큰 삭제 그는 발을 옮겼다. 아내는 멀어지는 그를 있었다. 그녀는 눈물을 . 그도 .었다. 문장 순열 그녀는 눈물을 흘렸다. 그는 정문으로 발을 옮겼다. 그의 아내는 멀어지는 그를 바라보고 있었다. 그도 마찬가지였다. 문서 회전 그를 바라보고 있었다. 그녀는 눈물을 흘렸다. 그도 마찬가지였다. 그는 정문으로 발을 옮겼다. 아내는 멀어지는 텍스트 채우기 그는 옮겼다. 아내는 멀어지는 그를 . 그녀는 눈물을 흘렸다. 그도 .

- 토큰 마스킹 : MLM과 동일한 기법, 입력문장의 일부 토큰을 마스크 토큰으로 치환
- 토큰 삭제 : 어떤 위치의 토큰이 삭제되었는지도 문제임 → 불필요한 정보 자동 필터링
- 문장 순열 : 마침표(.)를 기준으로 문장을 나눠 문장의 순서를 섞는 방법 → 순서 맞추기를 통해 단어들이 문장 내에서 어떻게 연결되는지 잘 파악
- 문서 회전 : 임의의 토큰으로 문서가 시작하도록 but 문장의 순서 유지 → 문장의 원래 시작 토큰을 맞춰야함 → 시작점을 인식하도록

- 텍스트 채우기 : 몇 개의 토큰을 하나의 구간(span)으로 묶고 일부 구간을 [MASK]로 대체 → 푸아송 분포의 델타를 3으로 설정해 계산

미세 조정 기법



- 문장 분류 작업 : 입력 문장을 인코더와 디코더 동일하게 입력/ 디코더의 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용
- 토큰 분류 작업 : 입력 문장을 인코더와 디코더 동일하게 입력/ 디코더의 각 시점별 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용

[BART가 BERT와 다른점]

- 추상적 질의응답 : 입력값을 조작해 출력 생성
- 문장 요약

→ BART의 사전 학습 방식과 유사해 성능이 뛰어남

모델 실습

1. 허깅 페이스의 datasets 라이브러리에서 뉴스 본문과 요약 텍스트 데이터셋을 가져온다.
2. BartTokenizer로 BART 입력 텐서 생성한다.
3. BartForConditionalGeneration로 모델 선언
 - BART 조건부 생성 클래스
 - 6개의 인코더와 디코더로 구성
4. 모델 학습 및 평가
 - N-gram의 기준으로 Rouge 점수를 활용

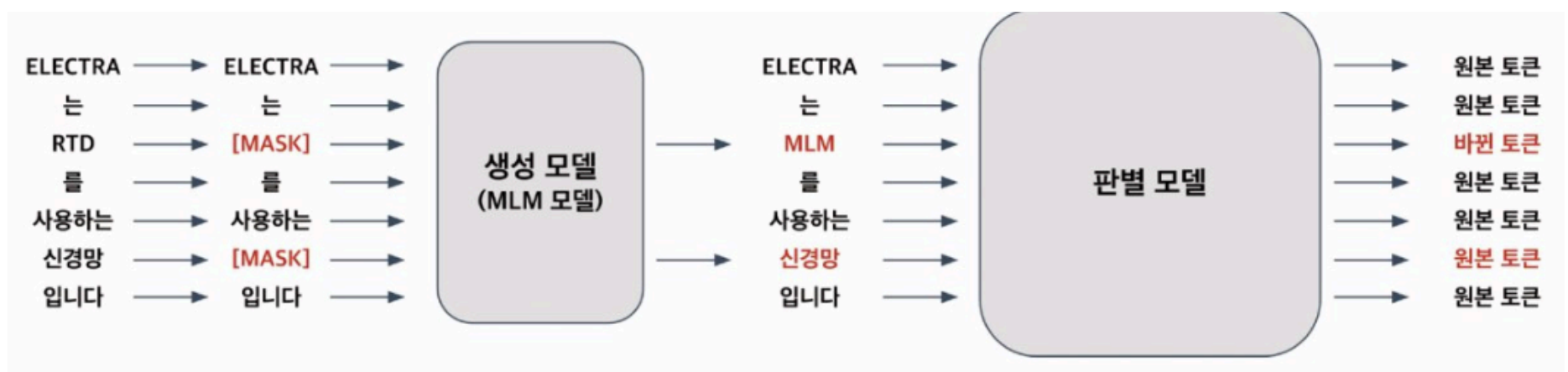
정답 문장 유니그램	[대한민국, 은, 16, 강, 에, 진출, 했다]
생성된 문장 유니그램	[대한민국, 은, 8, 강, 에, 진출, 하지, 못, 했다]
ROUGE-1 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$
ROUGE-1 정밀도	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$
정답 문장 바이그램	[대한민국은, 은 16, 16강, 강에, 에 진출, 진출했다]
생성된 문장 바이그램	[대한민국은, 은 8, 8강, 강에, 에 진출, 진출하지, 하지 못, 못 했다]
ROUGE-2 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$

ELECTRA

ELECTRA

- **ELECTRA** : 입력을 마스킹하는 대신 생성자와 판별자를 사용해 사전학습을 수행하는 트랜스포머 기반의 모델
- 사전 학습 방식 : 생성자와 판별자를 학습 → 생성적 적대 신경망 GAN과 유사한 방법으로 학습 수행
- 생성된 모델 : 실제 데이터와 비슷하게 토큰을 생성, 다른 토큰으로 대체, 판별 모델이 생성 모델이 만든 데이터와 실제 데이터를 입력 받아 어떻게 실제인지 판별
- BERT보다 매개변수 수가 더 적다

사전 학습 방법



- 생성자 모델 : 입력 문장의 일부 토큰을 마스크로 처리&예측하며 학습
↳ BERT의 마스킹된 언어 모델과 동일
- 판별자 모델 : RTD → 각 입력 토큰이 원본 문장의 토큰인지 생성자 모델로 바뀐 토큰인지 맞히면서 학습
↳ GAN과 유사하지만 차이점이 존재
 - ELECTRA는 생성 모델이 정확히 예측한 토큰을 원본 토큰으로 인식 not a 생성된 토큰
 - 완전 노이즈 벡터 <> 생성 모델의 일부 토큰이 마스크 처리된 텍스트를 입력

모델 학습

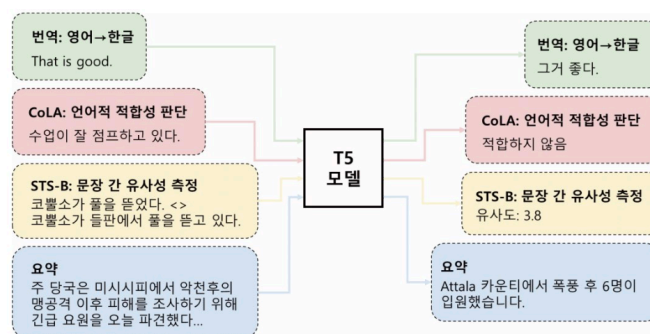
1. BERT 모델의 네이버 영화 리뷰 감정 분석 데이터세트로 학습
2. ElectraTokeniser 클래스로 토큰라이저를 불러옴
3. ElectraForSequenceClassification로 모델 선언
 - 토큰 임베딩 + 위치 임베딩 + 문장 구분 임베딩으로 구성 + 정규화 & 드롭아웃
4. 모델 학습 및 평가

T5

T5

- 인코더-디코더 모델 구조를 바탕으로 입출력을 모두 토큰 시퀀스로 처리하는 텍스트-텍스트 구조(<> 대부분의 입력 문장을 벡터나 행렬로 변환 → 출력 문장 생성)
- 텍스트-텍스트 구조로 입출력의 형태가 자유로움

[T5의 모델 학습 유형]



: 입출력이 모두 토큰(text) 시퀀스이기 때문에 입출력간의 관계를 세밀하게 다룰 수 있음

- CoLA 데이터셋 : 문법적으로 허용 or not 문장 구분 가능
- STS-B 데이터셋 : 문장 간 의미적 유사성 측정 가능
- C4 데이터셋 : 자연어 처리 작업 다양하게 하도록 사전 학습됨

[사전 학습 방식]

- 비지도 학습 방식 : 입력 문장의 일부를 마스킹 → 입력 시퀀스 처리 / 출력 시퀀스는 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성
- 센티널 토큰 <extra_id_n>이 마스크 토큰을 의미 → 이걸 예측하는 것이 목적

모델 실습

1. 허깅 페이스의 datasets 라이브러리에서 뉴스 본문과 요약 텍스트 데이터셋을 가져온다.
2. T5Tokenizer로 BERT와 비슷하게 사전 학습된 토큰라이저를 가져와 전처리 수행한다.
3. T5ForConditionalGeneration로 모델 선언
 - T5 조건부 생성 클래스
4. 모델 학습 및 평가